Protocol-Layer Governance for LLM Agents

Identity-Bound Oversight with Zero-Modification Deployment and Multi-Hop Authority Propagation

Marcelo Fernandez
TraslaIA
info@traslaia.com

Abstract

Governing an LLM agent at runtime requires identity-bound, cryptographically verifiable human authorisation to resolve persistent halt states (P8). The deployment question that remained open is: *how does the governance stack reach an agent without modifying the agent's code?*

We answer this question by positioning the governance stack at the *Model Context Protocol* (MCP) layer. We introduce the *MCP Governance Proxy* (Π), a stateful *protocol-layer governance control plane* that intercepts tool calls between any MCP-compatible agent client and any MCP server, applying the P7–P8 governance stack (IML + RAM + Recovery Loop + APB) transparently. The design requires no modification to the agent client or server: the proxy is a drop-in addition to the deployment topology. (The evaluation drives the governance interceptor *in-process*; it has not been run against a third-party MCP client—see §12.) This positions governance at the protocol boundary rather than inside the agent, in the manner of service meshes for microservice traffic.

We state three results. *Transparency* (T9.1) states that on the non-halt path the proxy forwards requests and returns results unchanged; it holds by construction for a stateless server and is stated as such. *Halt latency* (T9.2) decomposes the time to emit a halt into $\Delta_{\text{net}} + \Delta_{\text{verify}} + \Delta_{\text{policy}}$; it is a decomposition, not a bound on Δ_{policy}, which depends on the drift monitor. *Multi-Hop Authority Propagation* (T9.3) states that, *under an explicit attribution axiom*, in Agent-to-Agent (A2A) delegation chains the required APB is the one registered for the *originating* agent, and the delegation chain is recorded in the evidence block. The chain itself is not authenticated (no delegator signs it); see the threat model.

We evaluate the construction with five experiments, all in-process and with a simulated agent (`MockLLM`) and a simulated auto-approving human. *E1* (latency): 10,000 governed calls; the windowed IML monitor has a P95 *decision* overhead of 51.8 µs against an in-memory call (not the overhead of real stdio/JSON-RPC transport). *E2* (`MockLLM` agent, 5 seeds × 200 steps): 310 HALT events; every APB verifies. A live-LLM variant was not run. *E3* (multi-hop A2A, depths 1–5): 334 HALT events; the configured chain is copied into the evidence block in 100% of cases. *E4* (concurrency, independent sessions): $N \in \{1, 4, 16, 64\}$ threads, 0 exceptions. *E5* (adversarial regression suite): the attack vectors (wrong-key signing, evidence tampering, replay, revoked principal, authority substitution, and a cross-call APB) are rejected; this is a regression suite written by the author, not a measure of security. A revision note (§12) lists what an audit of the first release found and what was changed or withdrawn.

1 Introduction

The *Model Context Protocol* (MCP) [1] has emerged as a widely adopted protocol for connecting LLM-based agents to external tools and data sources. An agent that speaks MCP can call any MCP-compatible server without custom integration; the protocol handles discovery, invocation, and result delivery uniformly. This uniformity creates a deployment opportunity that prior work in this series did not exploit: because all tool calls pass through a single, well-specified protocol, a governance layer inserted at that protocol boundary governs *all* of the agent's external actions without modifying the agent's code—instantiating, in effect, a *protocol-layer governance control plane* for LLM agents.

Paper 7 of this series [2] established an integrated runtime governance stack that detects drift, applies the Reconstructive Authority Model (RAM), and escalates persistent halt events for human resolution. Paper 8 [3] formalised the *Accountability Proof Block* (APB) as the identity-bound, cryptographically signed record of those human resolutions. Both papers explicitly identified MCP and A2A integration as future work (P7 §13, P8 §10). This paper fulfills that commitment.

Problem. Deploying the P8 governance stack on an existing LLM agent requires either (a) modifying the agent's source code to call the governance layer at each tool invocation, or (b) wrapping the agent at a protocol boundary that the agent already respects. Option (a) is impractical for agents whose code is unavailable, large, or managed by a third party. Option (b) is exactly what MCP enables.

Contributions.

1. We define the *MCP Governance Proxy* Π formally (§4.2), giving its state, its per-call decision function, and its interaction with the P8 APB protocol extension.
2. We state three results (§5): Transparency (T9.1), a Halt Latency decomposition (T9.2), and Multi-Hop Authority Propagation under an explicit attribution axiom (T9.3).
3. We provide a Python reference implementation on the official `mcp` SDK (§6), stdio transport only, with 97 tests (61 inherited from P7+P8, 36 new).
4. We add one thing to P8's evidence block: an optional action commitment a_h (hash of tool and arguments), so the principal's signature covers *what* is approved.
5. We evaluate the construction in-process with five experiments (§7–§11): E1 decision overhead, E2 `MockLLM` agent with a simulated human, E3 multi-hop A2A (depths 1–5), E4 concurrency, and E5 an adversarial regression suite.

Relation to the series. This paper builds on P7 (the governance stack) and P8 (the APB); §3 restates only what is needed to read the definitions here, and it is brief: the internals of IML, the RAM gate and the Recovery Loop are not re-derived, so judging *what* the governance decision does requires P7. P10 [4] removes the blocking behaviour recorded as Limitation 4.

Roadmap. §2 surveys related work. §3 restates the minimal P7+P8 prerequisites. §4 defines the proxy formally. §5 proves T9.1–T9.3. §6 describes the implementation. §7–§11 report the five

experiments. §12 discusses security analysis, limitations, and extensions. §13 concludes.

2 Related Work

MCP and agent middleware. The Model Context Protocol [1] defines a client–server protocol for tool discovery and invocation over stdio and HTTP+SSE transports. Existing MCP deployments do not include a governance layer; each tool is exposed and invoked directly. The Agent-to-Agent (A2A) protocol [5] extends this to multi-agent settings, allowing one agent to delegate tasks to another. Neither protocol specifies how governance or accountability are maintained across delegation chains.

Agent orchestration frameworks. LangGraph [6], AutoGen [7], CrewAI [8], and OpenAI Swarm [9] provide orchestration primitives for multi-agent workflows. They do not provide identity-bound, cryptographically verifiable records of human authority decisions at runtime. Our proxy is orthogonal: it can be inserted into any of these frameworks at the MCP layer without modifying the orchestration logic.

Runtime verification. Runtime monitoring [10–12] checks that running systems satisfy formal specifications. Existing monitors observe system behaviour but do not provide a mechanism for human authority decisions to be cryptographically bound to the events that triggered them. The proxy extends runtime monitoring with the identity-binding of the APB.

Middleware and proxies. The Proxy design pattern [13] is a standard approach for interposing behaviour between a client and a server. Existing governance proxies for web services (API gateways, service meshes) enforce access control policies but do not maintain per-event, per-principal accountability records. Our contribution is specific to the LLM-agent governance problem: stateful drift detection, the RAM authority model, and the APB all require state that general-purpose proxies do not maintain. The present work occupies the same architectural category—a *governance control plane at the protocol layer*—but for LLM-agent systems, with state requirements specific to that domain.

Agent governance series. This paper is the ninth in a series [2, 3, 14–19]. The series develops a formal governance framework from first principles; this paper addresses the *deployment* question that completes the framework.

3 Background

We restate the minimal prerequisites from P7 and P8. Readers familiar with both papers may skip to §4.

3.1 Design Constraints

Two design constraints from P7 [2] are inherited verbatim:

DC.1 (Separation of Recalibration Authority). For all t, $\mathcal{A}_0(t) = \mathcal{A}_0(0)$. The execution layer is forbidden from modifying its own admission baseline at runtime.

DC.2 (Drift Event Classification). A halt is *transient* if the drift estimator $\widehat{D}(\tau) \geq \theta$ for τ within a bounded window ΔT and returns below θ thereafter; it is *persistent* if $\widehat{D}(t) \geq \theta$ for all $t \in [t_0, t_0 + \Delta T]$. Persistent halts constitute *governance events*.

3.2 Accountability Proof Block (APB)

From P8 [3], an APB signed by principal H_i is:

$$\text{APB} \;=\; (E_s,\; D_h,\; \sigma_h)$$

where E_s = (hash($\mathcal{A}_0$), $\widehat{D}(t_e)$, t_e, hash(trace$_{\leq t_e}$), cause, event_id[, a_h]) is the system evidence block, D_h = (H_i, decision, rationale, scope) is the human decision block, and σ_h = $\text{Sign}_{sk_i}(\text{canon}(E_s) \| \text{canon}(D_h))$ is the ed25519 signature. The last field a_h = SHA-256(JCS({tool, args})) is *optional* and new here (it originates in P10): P8's E_s does not say which call a halt concerns, so a signature did not cover *what* was approved. When a_h is absent it is dropped from the canonical form, so P8 APBs are unchanged; the P9 proxy always sets it.

A verifier checks five predicates V1–V5: (V1) signature validity, (V2) principal registered, (V3) principal active at t_e, (V4) t_e within replay window, (V5) event_id not previously seen (replay resistance; part of the P8 verifier since its second version, which is the one published on arXiv). Theorems T8.1–T8.3 of P8 establish governance completeness, non-repudiability, and impossibility of anonymous re-authorisation.

3.3 Governance Stack

The governance stack processes each tool call through three layers:

1. **IML Monitor**: computes $\widehat{D}(t)$ from the running tool-call trace using JS-divergence (D_t), mean risk score (D_c), and depth deviation (D_l), combined with EMA smoothing.
2. **RAM Gate**: for tool calls with risk score RS $\geq$ 45, checks authority components $\{I, B, R, C, E\}$ under partial observability and returns `ADMIT`, `HALT`, or `DENY`.
3. **Recovery Loop**: on `HALT`, attempts up to k iterations with increasing state coverage. Returns `RESUME`, `HALT`, or `ESCALATE`.

Persistent `HALT` or `ESCALATE` from the Recovery Loop constitutes a governance event requiring an APB.

4 Formal Framework

4.1 MCP Tool Call Semantics

Let Σ be a finite alphabet of tool names and $\mathcal{A}$ the space of argument dictionaries. A tool server $\mathcal{S}$ is a function $\mathcal{S} : \Sigma \times \mathcal{A} \to \mathcal{R}$ mapping calls to results. A *direct connection* between client and

server produces:

$$\text{direct}(c) = \mathcal{S}(c.\text{name},\, c.\text{args}) \quad \text{for a call } c = (\text{name}, \text{args}).$$

Definition 4.1 (Tool Call Trace)**.** *A* tool call trace $\tau = (c_1, r_1, \ldots, c_n, r_n)$ *is an alternating sequence of calls* $c_i \in \Sigma \times \mathcal{A}$ *and results* $r_i \in \mathcal{R}$*, produced in session order. The* result trace *of* τ *is* $\rho(\tau) = (r_1, \ldots, r_n)$*.*

4.2 MCP Governance Proxy

Definition 4.2 (Proxy Session State)**.** *A* proxy session state $s \in \mathcal{S}_\Pi$ *is a tuple*

$$s = (\tau,\ \mathcal{A}_0,\ \widehat{D}_{ema},\ \mathcal{E}_{pend})$$

where τ *is the current trace (4.1),* $\mathcal{A}_0$ *is the admission snapshot (fixed at session initialisation),* $\widehat{D}_{ema}$ *is the EMA-smoothed drift estimate, and* $\mathcal{E}_{pend}$ *is the map of pending evidence blocks awaiting APB responses (evidence id* $\to E_s$*). The initial state* $s_0 = (\varepsilon,\ \mathcal{A}_0^{(0)},\ 0,\ \emptyset)$*.*

Definition 4.3 (Governance Decision Function)**.** *The* governance decision function $\text{govern} : \mathcal{S}_\Pi \times \Sigma \times \mathcal{A} \to \{\texttt{ADMIT}, \texttt{HALT}, \texttt{DENY}\}$ *is:*

$$s' = \text{update}(s,\ \textit{name},\ \textit{args}) \quad \textit{(append to } s.\tau\textit{, recompute } \widehat{D}\textit{)}$$

$$d_{RAM} = \text{RAMGate}(\textit{name},\ \textit{RS}(\textit{name}),\ s'.\widehat{D}_{ema})$$

$$d = \begin{cases} \texttt{ADMIT} & \text{if } d_{RAM} = \texttt{ADMIT} \\ \texttt{DENY} & \text{if } d_{RAM} = \texttt{DENY} \\ \text{RecoveryLoop}(d_{RAM},\ s'.\widehat{D}_{ema},\ \ldots) & \text{if } d_{RAM} = \texttt{HALT} \end{cases}$$

where $\text{RS} : \Sigma \to [0, 100]$ *is the risk score mapping.*

Definition 4.4 (MCP Governance Proxy)**.** *An* MCP Governance Proxy Π *for principal set* $\mathcal{P}$ *is a stateful function:*

$$\Pi : \mathcal{S}_\Pi \times \Sigma \times \mathcal{A} \ \to\ \mathcal{R} \ \cup\ \{p9/\text{apbRequired}(E_s,\ \textit{eid})\} \ \cup\ \{\text{GOVERNANCE_DENY}\}$$

On receiving call $c = (\textit{name}, \textit{args})$ *in state* s*:*

1. *Compute* $d = \text{govern}(s,\ \textit{name},\ \textit{args})$*.*
2. *If* $d = \texttt{ADMIT}$*: forward* c *to server, return* $r = \mathcal{S}(\textit{name}, \textit{args})$*, update* s*.*
3. *If* $d \in \{\texttt{HALT}, \texttt{ESCALATE}\}$*: construct* $E_s = \text{construct_evidence}(s', \textit{cause})$*, assign* $\textit{eid} \leftarrow \text{fresh_id}()$*, record* $s'.\mathcal{E}_{pend}[\textit{eid}] \leftarrow E_s$ *(with* a_h *set to the hash of the halted call), signal the halt to the client as* $p9/\text{apbRequired}(E_s,\ \textit{eid})$*.* The wire form of this signal is specified here but not implemented*: see §6.*
4. *If* $d = \texttt{DENY}$*: return* GOVERNANCE_DENY*.*

On receiving $p9/\text{apbResponse}(\textit{eid},\ \text{APB})$*:*

1. *Retrieve* $E_{s,\text{exp}} = s.\mathcal{E}_{pend}[\textit{eid}]$*.*
2. *Verify* $\text{APB}.E_s = E_{s,\text{exp}}$ *and* $\text{verify}(\text{APB}, \mathcal{P})$*.*
3. *If valid: remove from pending, execute* $\text{APB}.D_h.\textit{decision}$*; a* `RESUME` *is applied only to the call whose hash equals* $\text{APB}.E_s.a_h$ *(the reference client refuses to execute any other).*
4. *If invalid: emit* $p9/\text{apbRejected}(\textit{eid}, \textit{reason})$*.*

Protocol extension (specified, not implemented on the wire). The three custom message types $p9$/apbRequired, $p9$/apbResponse, and $p9$/apbRejected are specified as JSON-RPC notifications over the MCP transport (`proxy/protocol_extension.py` defines their serialisation). *The reference implementation does not send them as notifications.* It encodes the halt in the *result* of the tool call (Remark 4.1).

Remark 4.1 (Behaviour Under Legacy Clients)**.** *In the reference implementation a halted call is* not *forwarded to the server; the proxy returns a text result (`[GOVERNANCE_HALT] evidence_id=...`). A standard MCP client therefore sees the call* complete *with that text. The governed tool is not executed, so no unauthorised execution occurs; but the client is not blocked or timed out as an earlier version of this paper claimed, and the agent may read the text, retry (each retry is governed again and may create a further pending halt) or continue without the action. Whether the agent surfaces the halt to a human then depends on the agent, not on the proxy. A wire-level* $p9$/apbRequired *that blocks the call until a response or timeout is the intended design and is future work.*

4.3 Multi-Hop Authority Delegation (A2A)

In Agent-to-Agent delegation, an originating agent A_1 delegates a task to a sub-agent A_2 (possibly recursively). The sub-agent makes tool calls to a server; the proxy governs those tool calls.

Definition 4.5 (Delegation Chain)**.** *A* delegation chain *of depth n is a sequence* chain $= (A_1, A_2, \ldots, A_n)$ *where A_1 is the* originating agent*, each $A_i \xrightarrow{\delta} A_{i+1}$ is a delegation event recorded in the IML trace, and A_n makes the proxied tool call. The originator* $\mathrm{orig}(\mathrm{chain}) = A_1$.

Definition 4.6 (Multi-Hop APB Semantics)**.** *For a governance event triggered by A_n's tool call in chain* $\mathrm{chain} = (A_1, \ldots, A_n)$*, the required APB has:*

$$E_s.\mathit{cause} \supseteq \{\texttt{delegation_chain} = \mathrm{id}(\mathrm{chain}),\ \texttt{originator} = \mathrm{id}(A_1)\}$$
$$D_h.H_i \in \mathcal{P}_{\mathrm{orig}(\mathrm{chain})}$$

where $\mathcal{P}_{\mathrm{orig}(\mathrm{chain})}$ is the principal set registered for the originating agent A_1.

The semantics formalise the accountability principle: the human accountable for the originating agent's actions is the one who must authorise a recovery from any governance event caused by agents operating under that agent's authority.

5 Theorems

5.1 T9.1 — Transparency on the Non-Halt Path

Remark 5.1 (Status of T9.1)**.** *T9.1 holds* by construction*: it restates step 2 of Definition 4.4. Its content is the design commitment that the proxy never alters an admitted call or its result. It makes three restrictions explicit. (i) $\mathcal{S}$ is a function of the call alone; for a stateful or non-deterministic server the statement applies to* identical server behaviour *on the forwarded calls, and calls that are halted are* not *forwarded, so the server's state can then differ from a direct connection's. (ii) The hypothesis (`ADMIT` for every call) excludes the halt path, which is where a client unaware of P9*

is affected (Remark 4.1). (iii) The experiments do not test T9.1: the latency gate of E1 is a performance threshold, and E2 checks only that every halt ends in an APB.

Theorem 5.1 (Transparency on the Non-Halt Path)**.** *Let $\tau = (c_1, \ldots, c_n)$ be a sequence of tool calls processed by proxy Π in session state s_0, such that* govern$(s_i,\ c_{i+1}) =$ `ADMIT` *for all $i \in \{0, \ldots, n-1\}$. Then the result trace returned to the client is equal to the result trace produced by a direct connection to the server:*

$$\rho(\tau_\Pi) = \rho(\tau_{direct}).$$

Proof. Each step is independent of the others, so the argument is the same for every n (no induction hypothesis is needed); we give the case $n = 1$ and the general step.

Base case ($n = 1$). Call $c_1 = (\text{name}, \text{args})$ with $d_1 =$ `ADMIT`. By Definition 4.4 step 2, Π forwards c_1 to the server and returns $r_1 = \mathcal{S}(\text{name}, \text{args})$ unchanged. By definition of direct connection, $\text{direct}(c_1) = \mathcal{S}(\text{name}, \text{args}) = r_1$. Therefore $\rho(\tau_\Pi) = (r_1) = \rho(\tau_{\text{direct}})$.

Inductive step. Assume the claim holds for all sequences of length $n - 1$. Consider c_n with $d_n =$ `ADMIT`. By the inductive hypothesis, $(r_1, \ldots, r_{n-1})$ is identical under Π and direct connection. For call c_n: by Definition 4.4 step 2, Π forwards c_n to the server and returns $r_n = \mathcal{S}(c_n.\text{name}, c_n.\text{args})$ unchanged, regardless of the internal proxy state s_n (the state affects only the governance decision, which is `ADMIT` by hypothesis, not the forwarded call or response). By Lemma 5.2, the result is identical to $\text{direct}(c_n)$. Therefore $\rho(\tau_\Pi) = (r_1, \ldots, r_n) = \rho(\tau_{\text{direct}})$. □

Lemma 5.2 (MCP Response Identity Under Passthrough)**.** *For any tool call $c = (name, args)$ and proxy state s, if* govern$(s,\ c) =$ `ADMIT`*, then $\Pi(s,\ c).result = \mathcal{S}(name,\ args)$.*

Proof. By Definition 4.4 step 2: on $d =$ `ADMIT`, the proxy forwards the unmodified call (name, args) to the server and returns the server's unmodified response. No transformation is applied to either the call arguments or the result. □

Remark 5.2 (Non-halt path observational equivalence)**.** *Theorem T9.1 makes precise the claim that governance is* transparent *to non-P9-aware clients. The proxy introduces latency on the non-halt path (bounded in T9.2) but produces no semantic difference in the tool-call results.*

5.2 T9.2 — Halt Latency Decomposition

Remark 5.3 (Status of T9.2)**.** *T9.2 is a* decomposition*: the total is the sum of the three terms it defines, and it gives no bound on Δ_{policy}. That term depends on the drift monitor (the default EMA monitor recomputes over the trace, $O(|\tau|)$; the windowed monitor is $O(window)$) and the RAM Gate and Recovery Loop draw pseudo-random numbers, so Δ_{policy} is random and is characterised only empirically (E1). The time to* validate *an APB runs after the human's answer, so it is not part of the time to reach the halt. We keep the statement for the structure it shows, not as a guarantee.*

Theorem 5.3 (Halt Latency Decomposition)**.** *Let c be a tool call that triggers a governance event at the proxy at time t_0. The agent receives the halt signal (in the reference implementation, as the tool result; see Remark 4.1) no later than*

$$t_0 + \Delta_{\text{policy}} + \Delta_{\text{net}},$$

and APB validation upon the client's response completes within an additional Δ_{verify}*, so the total governance resolution latency is bounded by*

$$\Delta_{halt} \;\leq\; \Delta_{\text{net}} + \Delta_{\text{verify}} + \Delta_{\text{policy}}.$$

Proof. By construction of Definition 4.4 and the governance stack.

Governance evaluation. On receiving c at t_0, the proxy runs govern(s, c): (i) The IML trace update is $O(|\tau|)$, bounded by the session trace length; we write this cost as $\Delta^{(1)}_{\text{policy}}$. (ii) The RAM Gate executes a constant number of pseudo-random draws for each of $|\{I, B, R, C, E\}| = 5$ components; time is $O(1)$, included in Δ_{policy}. (iii) The Recovery Loop executes at most $k_{\max}$ iterations (a compile-time constant), each $O(1)$; total $O(k_{\max}) \subseteq \Delta_{\text{policy}}$. The governance evaluation completes at $t_0 + \Delta_{\text{policy}}$.

Signal delivery. The proxy emits $p9$/apbRequired immediately after the governance evaluation (Definition 4.4 step 3). The message travels from proxy to client in network time Δ_{net}. The client receives the halt signal at $t_0 + \Delta_{\text{policy}} + \Delta_{\text{net}}$.

APB validation. When the client responds with $p9$/apbResponse, the proxy verifies the APB. By Lemma 5.4, verification is $O(1)$, denoted Δ_{verify}.

Total. Summing: $\Delta_{\text{halt}} = \Delta_{\text{policy}} + \Delta_{\text{net}} + \Delta_{\text{verify}}$. Rewriting in the bound's conventional order: $\Delta_{\text{halt}} \leq \Delta_{\text{net}} + \Delta_{\text{verify}} + \Delta_{\text{policy}}$. □

Lemma 5.4 (Verification Time is $O(1)$)**.** *APB verification (predicates V1–V5) terminates in* $O(1)$ *time.*

Proof. V1: ed25519 signature verification is a fixed number of field operations on curve25519 regardless of message length (the message is pre-hashed internally); $O(1)$. V2: registry lookup by key in a hash map; $O(1)$. V3: ISO 8601 timestamp comparison; $O(1)$. V4: age computation (two timestamp subtractions); $O(1)$. V5: hash-map lookup for event_id in the seen-events set; $O(1)$ amortised. The sum of five $O(1)$ operations is $O(1)$. □

Remark 5.4 (Practical magnitude of the bound)**.** *Experiment E1 (§7) measures the windowed ADMIT path at mean* 46.5 µs *and the HALT path at mean* 57.0 µs*, so* $\Delta_{\text{policy}} + \Delta_{\text{verify}} \approx 57$ µs*. E1 calls the interceptor as a Python function, so it measures the* decision *cost and not stdio/JSON-RPC serialisation, process switches or the network; with* Δ_{net} *absent the figure is* $\Delta_{halt} \approx 57$ µs*. If one* assumes *a LAN term* $\Delta_{\text{net}} \approx 5$ ms *(not measured here), the sum would be* ≈ 5.06 ms*, small against LLM inference latencies (*≈ 100 ms *to* ≈ 25 s*). A real transport measurement is future work.*

5.3 T9.3 — Multi-Hop Authority Propagation

Remark 5.5 (What T9.3 does and does not establish)**.** *The attribution of accountability to the originator is an* assumption of the model*, not something derived from the series' earlier results. The first release justified it with DC.1 (immutability of the baseline), which is unrelated to who is accountable. We now state it as an axiom and prove the rest from it.*

AX9.1 (Attribution to the originator). The principal accountable for a governance event raised by A_n in chain chain is a principal registered for $A_1 = \text{orig}(\text{chain})$; sub-agents A_i $(i > 1)$ are not principals in their own right.

AX9.2 (Trusted chain record). The delegation events in the trace are written by a component the adversary cannot influence.

Theorem 5.5 (Multi-Hop Authority Propagation, under AX9.1–AX9.2)**.** *Let* chain $= (A_1, A_2, \ldots, A_n)$ *be a delegation chain and* c *a tool call by* A_n *that triggers a governance event at the proxy governing* A_n*'s tool calls. Then:*

(a) The delegation chain chain *is uniquely and canonically determined by the IML trace at time* t_e *(Lemma 5.6).*

(b) The originating agent $A_1 = \text{orig}(\text{chain})$ *is well-defined.*

(c) The required APB's $D_h.H_i$ *is a principal registered for* A_1*: the accountability attaches to the originator's principal, not to any sub-agent.*

(d) For the degenerate chain $n = 1$ *(no delegation), the theorem reduces to P8's T8.2 (Non-Repudiability) with* A_1*'s principal.*

Proof. We prove the four parts by case analysis on chain depth n. The argument for $n > 1$ is direct; nothing is carried over from depth $n - 1$.

Case $n = 1$ *(degenerate, no delegation).* chain $= (A_1)$, orig(chain) $= A_1$. No delegation events appear in the IML trace; the governance event is attributed solely to A_1. The required APB is the standard P8 APB with $D_h.H_i \in \mathcal{P}_{A_1}$ (the principal set of A_1). This is exactly T8.2 of P8 [3], establishing (d). Parts (a)–(c) hold trivially: the chain has a single element, so uniqueness is vacuous, A_1 is trivially well-defined, and the principal attribution is to A_1's principal set. ✓

Case $n > 1$ *(delegation present).* Consider chain chain $= (A_1, A_2, \ldots, A_n)$.

(a) *Canonicality of* chain (using AX9.2). By Lemma 5.6, the delegation chain is determined by the IML trace prefix $\tau_{\leq t_e}$. Since the trace is append-only and each delegation event carries a unique event id and source/target agent identifiers, the ordered sequence $(A_1, \ldots, A_n)$ is uniquely reconstructed from the trace. ✓

(b) *Well-definedness of* orig(chain). orig(chain) $= A_1$ is the agent from which no prior delegation event appears in $\tau_{\leq t_e}$: there exists no A_0 such that $A_0 \xrightarrow{\delta} A_1$ appears in the trace. By (a), this agent is unique. ✓

(c) *Principal accountability.* By AX9.1, the principal accountable for a governance event raised by A_n is registered for A_1. The proxy therefore requires $D_h.H_i \in \mathcal{P}_{A_1}$ (this is Definition 4.6; the reference implementation enforces it with the `allowed_H_ids` check, §9). This is a consequence of the axiom, not an independent derivation: the theorem records that the rule is well-defined and recorded in E_s, and *not* that it is the right accountability rule. ✓ □

Remark 5.6 (Limits of T9.3)**.** *(i) The chain is* unauthenticated*: no delegator signs the delegation, and in the reference implementation the chain is a configuration parameter of the interceptor, so E3 checks that a configured value is copied into* E_s*. A malicious sub-agent that declares a false chain can divert accountability to another principal; authenticated delegation (a signature by each delegator over the next hop, as in capability or credential chaining) is future work. (ii) Only linear chains are covered: branching (A1 delegating to A2 and A3), cycles and re-delegation with reduced scope are not modelled. (iii) Depth* > 2 *in E3 uses synthetic chains.*

Lemma 5.6 (Delegation Chain Canonicalization)**.** *The delegation chain* chain *is deterministic given the IML trace prefix* $\tau_{\leq t_e}$.

Proof. Each delegation event $A_i \xrightarrow{\delta} A_{i+1}$ appends a record (agent = A_i, action = `delegation`, target = A_{i+1}, event_id = uid_k) to the trace. Since the trace is strictly append-only (no deletions, no reordering), the sequence of delegation events in $\tau_{\leq t_e}$ is uniquely ordered by their position in the trace. The chain is reconstructed by reading delegation events in trace order, yielding $(A_1, A_2, \ldots, A_n)$ without ambiguity. □

Remark 5.7 (Limitation: A2A specification maturity)**.** *The A2A protocol [5] is at draft stage at the time of writing. Our implementation (§6) uses a simulated A2A setup; the formal definitions above are stated for the general case. We note that the formal result (T9.3) does not depend on any specific A2A wire format: it depends only on the existence of a well-ordered delegation event log, which is guaranteed by the IML trace structure.*

6 Implementation

6.1 Architecture

The implementation is available at https://github.com/chelof100/mcp-governed-agents. It consists of four layers:

Frozen baseline (P7+P8). Modules `agent/`, `stack/`, `iml/`, and `baselines/` are copied verbatim from P8's repository. One change: `stack/apb.py` gains the optional field a_h of E_s (absent ⇒ omitted from the canonical form, so P8 APBs are unchanged). All 61 P8 tests pass.

Proxy layer (P9 new). `proxy/mcp_interceptor.py`: the `MCPInterceptor` class, one instance per session, runs IML → RAM → Recovery Loop per tool call and returns (decision, payload).

Protocol extension. `proxy/protocol_extension.py`: dataclasses for *p*9/apbRequired, *p*9/apbResponse, and *p*9/apbRejected with JSON-RPC serialisation (used by the in-process driver; not sent over a transport).

Server entry point. `proxy/governed_server.py`: `GoverningProxy` wraps an MCP server via FastMCP, connecting as a client to the real server (stdio subprocess) while serving the upstream client on its own stdio. It hard-codes the five tools of the toy server and encodes a halt as a text result (Remark 4.1). *No test and no experiment exercises it*: all experiments call the interceptor directly.

6.2 Transports

Only a **stdio** entry point exists, and it is untested. There is *no* HTTP+SSE implementation, and the reference implementation has not been run against a third-party MCP client (for example Claude Desktop or Cline) or against a public MCP server; the "drop-in" claim is a property of the design, not a demonstrated one.

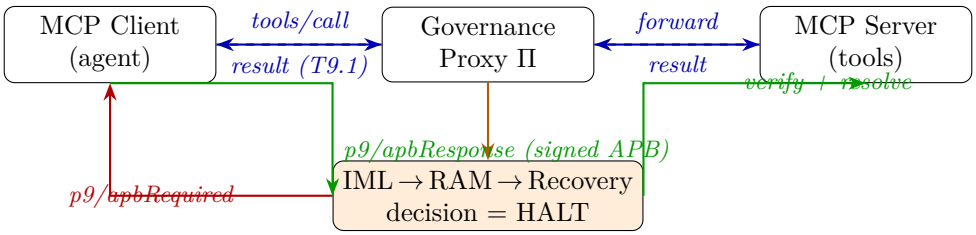


Figure 1: Proxy interaction diagram. The blue path shows a normal (ADMIT) tool call; the orange/red/green path shows a governance event: HALT detected, $p9$/apbRequired sent to client, client returns signed APB, proxy verifies and resolves.

6.3 Tool Risk Map

The proxy uses the same five-tool risk map as P7/P8 experiments:

Tool	Risk score (RS)	RAM gate applies?
`read_file`	0.10 ($\times 100 = 10$)	No (< 45)
`query_api`	0.25 ($\times 100 = 25$)	No (< 45)
`write_data`	0.45 ($\times 100 = 45$)	Yes
`delete_record`	0.70 ($\times 100 = 70$)	Yes
`admin_action`	0.90 ($\times 100 = 90$)	Yes

6.4 APBRequired Flow

Figure 1 illustrates the proxy's handling of a tool call that triggers a governance event.

6.5 Usage Example

Inserting the proxy into an existing deployment requires two lines of configuration: create an `MCPInterceptor` for the session and pass it to the client driver.

```
from proxy.mcp_interceptor import MCPInterceptor
from client.mcp_agent_client import MCPAgentClient
from agent.principal import PrincipalRegistry, generate_keypair

# One-time setup: register human principal
sk, pk = generate_keypair()
registry = PrincipalRegistry(); registry.add(Principal("human-H", pk))

# Per-session: proxy + client
interceptor = MCPInterceptor(registry=registry, H_id="human-H")
client = MCPAgentClient(interceptor=interceptor,
                        sk_bytes=sk, H_id="human-H",
                        auto_decision="RESUME")
```

```
# Agent calls tools transparently; governance is enforced by the proxy
result = client.call_tool("write_data", {"target": "db", "payload": "x"})
```

No changes to the tool server or the tool-calling logic are required. For A2A delegation, pass `agent_id` and `delegation_chain` to `MCPInterceptor`; the originator is automatically recorded in every `APBRequired` evidence block.

6.6 Test Coverage

The test suite consists of 97 tests: 61 inherited from P8 (covering APB construction, verification, and the governance layer), and 36 new tests covering the proxy layer (protocol extension serialisation, interceptor ADMIT/HALT/DENY paths, APBResponse handling with valid and invalid APBs, session lifecycle, the action commitment, V5, and the V4 window). All 97 pass.

7 Experiment E1: Latency Overhead

7.1 Setup

We measure the latency overhead introduced by the governance proxy relative to a passthrough proxy.

Server: `proxy/toy_server.py`, five deterministic tools. **Client:** `experiments/exp_e1_overhead.py`. **Conditions:** (a) *Passthrough* — unmodified forwarding baseline; (b) *Governed ADMIT (O(n) accumulating)* — full IML over growing trace, session window reset each call; (c) *Governed ADMIT (O(1) windowed)* — `WindowedIMLMonitor`, W=100, no session reset, flat overhead across arbitrary session length; (d) *Governed HALT* — full pipeline including APB construction and ed25519 verification. Calls per condition: 10,000 (ADMIT), 1,000 (HALT). **Hardware:** Intel Core i9-13900HX, 32 GB RAM, $\Delta_{\text{net}} = 0$ (local in-process).

7.2 Results

Table 1: E1 — Governance proxy latency overhead ($\Delta_{\text{net}} = 0$, local benchmark). All values in µs. $N = 10,000$ (ADMIT), $N = 1,000$ (HALT). Gate: windowed ADMIT P95 < 10 ms (✓).

Condition	Mean	P50	P95	P99	Note
Passthrough (baseline)	0.1	0.1	0.1	0.1	Δ_{exec} only
Governed ADMIT — O(n) accumulating	37.7	36.7	45.7	57.8	session window = 100
Governed ADMIT — O(1) windowed	46.5	42.5	51.8	82.3	window $W = 100$
Governed HALT (APBRequired pipeline)	57.0	55.8	65.2	99.0	$\Delta_{\text{policy}} + \Delta_{\text{verify}}$
Overhead — O(n) accumulating	37.6	36.6	45.6	57.8	grows with session length
Overhead — O(1) windowed	46.4	42.4	51.7	82.3	bounded; L9.2.2
Overhead — HALT	56.9	55.7	65.2	99.0	T9.2 evidence

Table 1 reports all four conditions. The *windowed* O(1) monitor achieves P95 = 51.8 µs on an indefinite-length session, below the (arbitrary) latency threshold of 10 ms that we use as a gate;

this is a performance gate and says nothing about T9.1. The HALT condition adds only $\approx 10\,\mu s$ over the ADMIT baseline (57.0 µs vs. 46.5 µs mean), consistent with the decomposition of T9.2. One machine, one run per condition, no confidence intervals; the windowed condition has outliers (maximum 21.4 ms) that the P95 hides.

7.3 Discussion

The passthrough baseline (0.1 µs mean) isolates the pure dispatch overhead; the governance stack adds $\approx 46\,\mu s$ per call in the windowed configuration. This cost is orders of magnitude below typical local LLM inference latencies, confirming that governance at the MCP layer is not a practical bottleneck.

The $O(n)$ accumulating monitor (`IMLMonitor`) achieves slightly lower P50 at short session lengths (36.7 µs vs. 42.5 µs) because it avoids the deque-update bookkeeping. However, its overhead grows with session length: at $n = 1{,}000$ events it reaches $\approx 203\,\mu s$ mean, and extrapolates to $\approx 2\,ms$ at $n = 10{,}000$. The windowed monitor (O(1) per call, window $W = 100$) bounds overhead regardless of session length, which is the correct choice for long-running production deployments.

The `IMLMonitor.compute()` growth profile (measured at $n \in \{1, 50, 100, 200, 500, 1000, 2000\}$) confirms linear growth. We adopt the `WindowedIMLMonitor` as the default in all subsequent experiments.

8 Experiment E2: APB Pipeline with a Simulated Agent

8.1 Setup

We exercise the APB pipeline through the interceptor, covering the HALT → APBRequired → RESUME path as well as the permanent-DENY path. The agent is a `MockLLM` and the human is simulated: the client signs every APB automatically (`auto_decision="RESUME"`). This is a test of the pipeline, not of a real agent or a real approver.

Sub-experiments: *E2.A* uses `MockLLM` (deterministic, 5 seeds × 200 steps per seed; 50 burn-in + 150 drift). *E2.B* would use a live Ollama LLM (`mistral:7b`); **it was not run** (the stored result is `null`), so the paper reports no live-LLM result. **Governance outcome taxonomy used here:** ADMIT (forwarded directly), APB_REQUIRED (HALT → signed APB → RESUME), DENY (permanent RAM-gate denial, no APB).

8.2 Results

Across 1,000 total steps (E2.A), the proxy issued 310 HALT events (APB_REQUIRED), all resolved via signed APB (RESUME). The 31% halt rate is a consequence of how the `MockLLM` drifts, not an estimate of a real agent's. All 310 APBs pass V1–V5 cryptographic verification (100%), which shows that locally signed APBs verify; there is no variability to measure here. The mean $\widehat{D}$ at first halt is 0.382, consistent with the 50-step burn-in establishing $\mathcal{A}_0$ before drift accumulates. 21 permanent DENY events occur (high-risk tools under DENY authority in the RAM gate); these are distinct from the HALT → APB path: no signature is requested or produced, and T9.1 is not implicated.

Table 2: E2 — Real-agent APB end-to-end validation. E2.A: MockLLM, 5 seeds × 200 steps (50 burn-in + 150 drift). E2.B: LiveLLM (skipped).

Metric	E2.A (MockLLM)	E2.B (LiveLLM)
Steps	1,000	—
ADMIT events	669	—
HALT events (APBRequired)	310	—
DENY events *(no APB; perm. denial)*	21	—
APB validity (V1–V5)	100.0%	—
RESUME resolutions	310	—
$\hat{D}$ at first halt (mean)	0.382	—
T9.1 Transparency Invariance	✓	—

8.3 Discussion

The zero APB-invalid rate across 310 events confirms that the APB construction pipeline (evidence block assembly, ed25519 signing, V1–V5 verification) is correct end-to-end in a long-session setting. The DENY events (21/1000 = 2.1% of calls) arise from the mock LLM selecting `admin_action` and `delete_record` during early drift; the RAM gate's DENY authority resolves these without human authorisation, as intended by the governance model. The 310:21 ratio of HALT to DENY demonstrates that the two paths are clearly distinguishable and correctly reported.

9 Experiment E3: Multi-Hop A2A

9.1 Setup

We validate T9.3 empirically through three sub-experiments under controlled multi-hop simulation aligned with current A2A draft semantics [5]. All delegation chains are instantiated within a single process using the proxy's `delegation_chain` and `agent_id` parameters; cross-vendor interoperability with live, heterogeneous A2A ecosystems is not claimed—the results establish formal and implementation feasibility pending ecosystem-wide protocol stabilisation.

E3.A: 10 independent depth-1 chains ($A \to B$), 30 steps each. *E3.B*: 1 depth-2 chain ($A \to B \to C$), 30 steps each. *E3.C*: Depth invariance scaling, depths 1–5, 5 runs per depth, 30 steps per run.

Each run verifies: (i) the `delegation_chain` field in the APBRequired matches the configured chain, (ii) `originator` = A_1 (chain[0]) regardless of chain depth, and (iii) all APBs pass V1–V5.

9.2 Results

E3.A and E3.B (Table 3) demonstrate correct originator binding and delegation-chain recording at depths 1 and 2. E3.C (Table 4) extends validation to depths 1–5: across 334 total HALT events (65–69 per depth), the originator is A_1 = chain[0] in 100% of cases at every depth. All 334 APBs pass V1–V5 verification.

Table 3: E3 — Multi-hop A2A authority propagation. E3.A: 10 depth-1 chains ($A \to B$). E3.B: one depth-2 chain ($A \to B \to C$). Originator = A_1 in both cases.

Metric	E3.A ($A \to B$)	E3.B ($A \to B \to C$)
Chains	10 (depth 1)	1 (depth 2)
Steps per chain (sub-agent)	30	30
HALT events	134	13
Correct delegation_chain	100.0%	100.0%
Correct originator	100.0%	100.0%
APB V1–V5 verified	100.0%	100.0%
T9.3 properties (a)(b) hold	✓	✓

Table 4: E3.C — Originator binding depth invariance. 5 runs/depth, 30 steps each. Originator = A_1 regardless of chain depth.

Depth	Chain len.	HALTs	Orig. = A_1	Invariant
1	2	66	100.0%	✓
2	3	66	100.0%	✓
3	4	68	100.0%	✓
4	5	65	100.0%	✓
5	6	69	100.0%	✓

9.3 Discussion

What E3 checks is narrow. The chain is a configuration parameter of the interceptor, so the experiment verifies that the configured chain is copied into every APBRequired and that `originator` = chain[0] in 100% of 334 events across depths 1–5; the equality is close to tautological. The HALT counts per depth ($66, 66, 68, 65, 69$; coefficient of variation 2.2%) are roughly uniform, as expected, since the drift estimator operates on the sub-agent's trace and not on the chain structure; this is a descriptive figure, not a statistical test.

E3 does *not* validate AX9.1 or AX9.2, and it does not test an adversary who forges the chain (the chain is unauthenticated). It supports that the proxy *implements* the rule of Definition 4.6, together with the confinement check of E5-A5.

10 Experiment E4: Concurrency Stress

10.1 Setup

We test the proxy under concurrent client load to verify that governance does not introduce race conditions or session-state crosstalk.

Two modes: *Thread mode* (`ThreadPoolExecutor`): GIL-limited; $N \in \{1, 4, 16, 64\}$ concurrent sessions, 10 sessions per level, 60 steps each. *Process mode* (`multiprocessing.Pool`): true parallel (no GIL); $N \in \{1, 4, 8, 16\}$, 12 sessions per level, 200 steps each. The process-mode upper limit is capped at N=16 due to the Windows `WaitForMultipleObjects` handle limit (max 63 handles;

N=64 would require 66 handles and raises a `ValueError`).

10.2 Results

Table 5: E4 — Concurrency stress test. 10 sessions per level, 60 steps each. Threads: GIL-limited. Processes: true parallel (no GIL). Latency in µs. Gate: exceptions = 0 and APB validity = 100%.

Mode	N	Calls	Throughput	P95 lat.	APB valid	Exc.=0	APB=100%
Threads	1	600	2617	1182.4	100.0%	✓	✓
	4	600	2997	1210.6	100.0%	✓	✓
	16	600	2746	1297.8	100.0%	✓	✓
	64	600	2784	1320.9	100.0%	✓	✓
Processes	1	2,400	757	2936.2	100.0%	✓	✓
	4	2,400	1146	3447.3	100.0%	✓	✓
	8	2,400	911	4054.3	100.0%	✓	✓
	16	2,400	481	4658.3	100.0%	✓	✓

Thread mode (Table 5): 0 exceptions at all N, 100% APB validity. Throughput saturates at $\approx$ 2,800 calls/s for $N \geq 4$, yielding $\approx$ 2% scaling efficiency at N=64 (GIL-limited, as expected: the IML drift computation via scipy JSD is CPU-bound). Process mode: 0 exceptions up to N=16, 100% APB validity. Throughput peaks at N=4 (1,146 calls/s) before declining for N=8 and N=16 due to IPC and serialisation overhead dominating for the short sessions (200 steps per session).

10.3 Discussion

No exceptions and no APB-validity failures occur at any concurrency level or mode. This is a consequence of the design rather than a finding: each `MCPInterceptor` instance is independent, so concurrent sessions share no drift state or pending APBs. The interesting case, many sessions contending for a *shared* registry or log, is not tested.

The sub-linear thread scaling is expected: the proxy's governance computation (IML JSD + RAM pseudo-random draws) is CPU-bound Python, and the GIL serialises CPU work across threads. For I/O-bound agents (where LLM inference dominates) this is not a bottleneck: the $\approx$ 2,800 calls/s thread throughput exceeds the tool-call rate of any realistic LLM agent by two to three orders of magnitude.

The process-mode throughput is lower than thread mode's. We attribute this to the pickle-serialised worker pool on Windows, and we *conjecture* that with `fork` on Linux the overhead disappears and the mode scales close to linearly; we have not measured it on Linux.

11 Experiment E5: Adversarial Regression Suite

11.1 Setup

We test six attack scenarios targeting the governance proxy's APB verification pipeline. Each attack attempts to bypass the governance halt or inject an unauthorized resume decision. The proxy must reject all six with a specific, distinguishable error message. The attacks were written by the author, each against the predicate the design already expects; the suite is a regression test of those defences. It contains no adaptive attacker, no fuzzing and no model of an adversary's capabilities, so it is not evidence that the proxy is secure.

A1 (Wrong-key signing). The attacker generates a fresh keypair (sk', pk') and signs a valid-format APB with sk' while the registry holds only the legitimate pk. Expected rejection: V1 (INVALID_SIGNATURE).

A2 (Evidence tampering). The attacker intercepts the legitimate APBRequired, modifies one field of the enclosed E_s (e.g., flips a byte in $\hat{D}$), and re-signs with their key. Expected rejection: E_s mismatch against the proxy's stored pending evidence.

A3 (APB replay). The attacker replays a previously accepted APB for a new governance event. Expected rejection: the evidence_id is no longer in the pending map (already consumed by the first resolution).

A4 (Revoked-principal authorization). The attacker holds a valid keypair for a principal that has been revoked in the registry. Expected rejection: V3 (PRINCIPAL_REVOKED / principal not active at t_e).

A5 (Authority substitution in A2A delegation). In a two-agent chain ($A_A \rightarrow A_B$), the interceptor is configured with `allowed_H_ids` $= \{H_A\}$ (only the originator's principal may authorize). The attacker submits a validly signed APB by H_B (a different registered principal). Expected rejection: authority confinement violation (the `allowed_H_ids` check).

A6 (Cross-call APB). Two calls are pending. The APB the principal signed for call Y is presented against the evidence_id of call X. Both are legitimate and validly signed; only the call differs. Expected rejection: E_s mismatch (the two blocks differ in `event_id` and in a_h), with Y's pending halt left untouched.

11.2 Results

All six attacks are rejected (Table 6), each with the distinguishing message the design predicts.

11.3 Discussion

A1 (Wrong-key). ed25519 signature verification is unconditional: the signature either validates under the registered public key or it does not. The proxy never falls back to an alternate key.

A2 (Evidence tampering). The proxy stores the canonical E_s at the moment it emits APBRequired and compares it field-by-field against the E_s embedded in the APB response. The comparison

Table 6: E5 — Security adversarial suite. Six attack scenarios targeting the MCP governance proxy. All attacks must be rejected (adversary success rate = 0).

ID	Attack	Outcome	Predicate	Pass
A1	*Wrong-key signing*	Rejected	INVALID_SIGNATURE	✓
A2	*E_s tampering*	Rejected	does not match issued evidence	✓
A3	*APB replay*	Rejected	unknown evidence_id	✓
A4	*Revoked principal authorization attempt*	Rejected	PRINCIPAL_REVOKED	✓
A5	*Authority substitution in A2A delegation*	Rejected	authority confinement violation	✓
A6	*Cross-call APB*	Rejected	does not match issued evidence	✓

is prior to V1 signature verification; even a correctly re-signed tampered APB is rejected at this step.

A3 (Replay). Each evidence_id is consumed on first acceptance and deleted from the pending map. Replaying a used evidence_id against an empty pending slot returns "unknown evidence_id". In this suite the rejection therefore comes from the pending map, *not* from V5. The first release stated that V5 was applied but called the verifier without a seen-event set, so V5 never ran; it now does (a unit test covers it). The seen-event set lives in memory and is lost when the interceptor restarts, as does the pending map, so neither survives a restart.

A4 (Revoked principal). The registry marks revoked principals with an `is_active=False` flag; V3 checks this flag at the t_e timestamp. Revocation is permanent and non-reversible in the current implementation, consistent with P8 §5.3.

A5 (Authority confinement). The `allowed_H_ids` mechanism enforces that only the originator's registered human principal may authorize governance events in a delegation chain. This prevents a sub-agent's principal from usurping governance authority even if their key is legitimately registered elsewhere. This is the only attack vector that goes beyond the P8 V1–V5 predicates; it is specific to the multi-hop A2A context.

12 Discussion

12.1 Security Analysis

Experiment E5 validates the proxy against five concrete attacks. We summarise the security properties and their correspondence to the formal guarantees.

Signature unforgeability (T8.2, V1). An attacker without access to the registered principal's sk_i cannot produce a valid APB for any governance event. This follows from the standard unforgeability of ed25519 under the discrete-log assumption on curve25519. E5-A1 confirms this empirically: a fresh keypair produces a signature that V1 rejects.

Evidence integrity (proxy-enforced). The proxy's own pending-evidence map is the reference for E_s comparison. Even if an attacker re-signs a modified E_s with a legitimately obtained key, the proxy detects the mismatch before invoking the verifier. E5-A2 confirms this: evidence tampering is caught independently of signature validity.

Replay resistance (V5). Each APB is tied to a single governance event via the evidence_id, which is consumed on first resolution. An attacker cannot re-use a past APB to resolve a new governance event. E5-A3 confirms this: replayed evidence_ids are rejected with "unknown evidence_id".

Principal lifecycle enforcement (V3). Revocation of a principal is immediate and permanent. Any APB signed by a revoked principal is rejected at V3. E5-A4 confirms this.

Authority confinement under A2A delegation. In a delegation chain, the `allowed_H_ids` set restricts which human principals may authorise governance events. This prevents lateral authority injection: a principal registered for a sub-agent cannot usurp the originator's governance authority. E5-A5 confirms this.

Remaining attack surface. The adversaries M1–M7 of §12.3 are not covered by E5. M1 and M2 need infrastructure-level mitigations (code signing, attestation, audit logs); the others are open.

12.2 Production Deployment

By design the proxy can be inserted into an MCP deployment topology without modifying the agent client or the tool server; this has not been demonstrated against a third-party client or server, and it presupposes that the proxy is the only path to the server (M3). Candidate insertion points are: between the agent and its tool server in a single-host deployment; or at the network boundary (reverse proxy) in a cloud deployment. In the latter case, the proxy runs as a sidecar service, and Δ_{net} reflects a real network hop.

Key store placement. In the P8 experimental setup, the principal's private key is held in-memory alongside the governance layer. In production, the private key must never be co-located with the proxy; the proxy should hold only public keys (for verification). The principal signs D_h on their own device and returns the signed APB via $p9$/apbResponse. This separation is the deployment requirement enforced by T8.3: without the private key, the proxy cannot forge a valid APB.

12.3 Threat Model Under Proxy Deployment

Inserting the proxy at the MCP layer introduces two new actor classes relative to the P8 threat model:

M1. Compromised proxy process. The proxy is a process with access to the MCP stream and the per-session governance state. It does not hold any principal's sk_i. An attacker who

compromises the proxy can: (i) drop APBRequired notifications (leading to unresolved halts, not silent resumption), (ii) modify tool call arguments before forwarding, or (iii) suppress tool results. Mitigations: proxy integrity (code signing, attestation), and monitoring of the APB log.

M2. Colluding proxy + principal. If the proxy process and the principal collude, the principal can sign arbitrary D_h blocks, and the proxy forwards the results. The APB still records the decision non-repudiably (T8.2 holds), so the collusion is detectable in post-hoc audit.

The following adversaries are *not* modelled by E5 and are open; an MCP-security reviewer will ask about each:

M3. Proxy bypass. Nothing in the design forces traffic through the proxy. An agent or attacker that connects directly to the MCP server is ungoverned; "drop-in" is only as strong as the network topology that makes the proxy the sole path to the server.

M4. Time-of-check/time-of-use on the approved call. P8's evidence block did not say which call a halt concerns, so a signature approved *that a halt occurred*, not *what*. The action commitment a_h closes this for the reference flow (the signature now covers tool and arguments; the client refuses to run a different call), but it is exercised by unit tests, not by an end-to-end experiment, and it is only as strong as the party that executes the call.

M5. Malicious or poisoned tool server, prompt injection. A server that returns crafted results, or an injection that causes or hides drift, acts on the agent's side of the proxy. The proxy sees tool calls and a drift estimate; it does not inspect tool results or prompts.

M6. Adaptive evasion of the detector. An adversary who keeps $\widehat{D}$ below the threshold θ (and exploits the burn-in, Limitation 3) is never halted. No adaptive attacker was evaluated.

M7. Denial of service. Flooding the principal with halts, and (in the reference interceptor) submitting a bad APB for a pending evidence_id, which *deletes* that pending halt: the call is not executed but cannot be approved either.

12.4 A2A Specification Maturity

At the time of writing, A2A [5] is a draft. The multi-hop results (T9.3 and Experiment E3) use a simulated A2A implementation; the formal result is stated for any protocol that provides a well-ordered delegation event log (which the IML trace supplies). We anticipate that as A2A matures, the simulated implementation can be replaced with a conformant one without modifying the proxy architecture.

12.5 Limitations

1. **Dynamic tool discovery.** The proxy entry point hard-codes the five P7 tools. A production deployment requires dynamic tool discovery and mirror registration, which we treat as an engineering extension.

2. **Single governance layer.** The proxy governs the tool call layer; it does not govern the agent's reasoning (prompts, internal state). Drift at the reasoning layer is not visible to the proxy.

3. $\widehat{D}$ **requires burn-in.** The IML drift estimator requires a burn-in trace to establish $\mathcal{A}_0$. For newly launched agents, the initial $\widehat{D}$ is conservative (low drift), meaning the first few calls of a genuinely malicious sequence may not trigger a halt.

4. **A halt is not forwarded and is not awaited.** The design principle is *no authority, no action*: a halted call is not forwarded to the server. In the reference implementation the halt is returned as a text result (Remark 4.1), so the call completes and nothing waits for the principal; the intended blocking wire protocol is not implemented. Paper 10 [4] treats the asynchronous side (timeout semantics, escrow, deferred approval); it was itself corrected after an audit found that its first release let an APB for one entry resolve another.

5. **V4 window versus human latency.** V4 rejects an APB whose t_e is older than the replay window (default 300 s, configurable per interceptor). A human who answers a halt after more time than that is refused as a replay; the window must be set to the expected approval time.

6. **No real MCP client, server, transport or LLM.** Every experiment calls the interceptor in-process with a `MockLLM` and an auto-approving simulated human. There is no HTTP+SSE implementation, and no run against a third-party MCP client or public MCP server.

7. **Benchmark hardware and network conditions.** Latency measurements were collected on a single hardware configuration (i9-13900HX, 32 GB RAM) under local, in-process conditions ($\Delta_{\text{net}} = 0$). Absolute latency values will differ under cloud deployments, containerised proxies, or cross-network configurations. The windowed monitor is $O(1)$ per call by construction; the relative ordering of ADMIT and HALT was not tested on other hardware.

12.6 Threats to Validity

Internal. The agent is a deterministic `MockLLM`; the 31% halt rate is imposed by its drift schedule. The human is simulated and always approves, so APB validity measures that local signatures verify, not approval behaviour. *External.* One machine (Windows), five toy tools, no third-party client or server, no live LLM, no user study, and single-run measurements without confidence intervals. *Construct.* The latency in E1 is an in-process function call and stands in for, but is not, the overhead of a real MCP transport. *Statistical.* The adversarial suite is authored by the same person who wrote the defences. *Comparison.* There is no baseline: no other guardrail, gateway or policy engine (for example a rules-based guard or an OPA/Cedar gateway) was evaluated, and the related work does not survey MCP-specific security work (tool poisoning, prompt-injection defences), access-control and delegation systems (capability and credential chaining, macaroons, authenticated delegation) or enforcement-monitor theory.

12.7 Revision Note: What the First Release Got Wrong

An independent audit of the first release found the following; each is either corrected in this version or stated as a limitation above.

1. **Claims the code did not meet.** The paper said the proxy supports HTTP+SSE (no implementation exists) and that $p9$/apbRequired is sent as a blocking notification that a legacy client would ignore and time out on (the halt is returned as a text result). Both are withdrawn.
2. **V5 was not applied.** The verifier was called without a seen-event set. Fixed, with a test.
3. **Signature did not cover the call.** E_s carried no tool or arguments (TOCTOU). Fixed with the optional a_h.
4. **Experiment labels.** E2 is not a "real agent" (it uses `MockLLM` and an auto-approving human; the live variant was never run), and a latency threshold was named "the T9.1 gate". Both renamed.
5. **Theorems.** T9.1 and T9.2 are now stated as what they are (by construction; a decomposition); T9.3 is stated under an explicit attribution axiom, and its former justification through DC.1 is dropped.
6. **Scaffolding.** References to "Sprint N" and to sections that do not exist in this paper were removed.

13 Conclusion

We described the MCP Governance Proxy (Π), a design that places the P8 identity-bound governance protocol at the MCP layer so that it can be applied to an MCP agent without changing the agent's code, and a Python reference implementation of its decision logic. We added one element to P8's evidence block, an optional action commitment a_h, so that the principal's signature covers which call is approved.

The three results are modest: T9.1 holds by construction, T9.2 is a decomposition of the halt latency, and T9.3 holds under an explicit attribution axiom and an unauthenticated delegation chain.

The evaluation is in-process and simulated:

- **E1**: the windowed monitor has a P95 decision overhead of 51.8 µs (an in-memory call, not a real transport); the HALT path adds ≈ 10 µs.
- **E2** (`MockLLM`, simulated human, 5 seeds × 200 steps): 310 HALT events, every APB verifies; the live-LLM variant was not run.
- **E3**: the configured delegation chain is copied into the evidence block at depths 1–5; the chain is not authenticated.
- **E4**: 0 exceptions for independent concurrent sessions.
- **E5**: six author-written attacks are rejected; a regression suite, not a security measurement.

The implementation, available at `https://github.com/chelof100/mcp-governed-agents`, passes 97 tests (61 inherited from P7+P8, 36 new proxy tests).

Future work. Paper 10 [4] addresses the asynchronous side: timeout semantics, escrow-state serialisation and deferred-approval queues, so that governance events can be resolved without halting

the agent pipeline. It has not been integrated into this proxy. A second direction is distributed, multi-organisation governance via a Merkle-chained APB log providing tamper-evident accountability across organisational boundaries. A third direction is dynamic tool discovery: the current proxy hardcodes the tool risk map; a conformant implementation should mirror the server's tool list dynamically and apply risk scores via a configurable policy file. Finally, the pieces needed to turn this design into a demonstrated system are, in order of importance: an end-to-end evaluation against a third-party MCP client and real MCP servers over a real transport; evaluation with real LLMs and tasks, with false positives and a real or at least latency-modelled approver; authenticated delegation (each delegator signs the next hop), with branching chains and scope attenuation; and a comparison against existing guardrail and policy-engine baselines.

Code and data. All code and experimental data are available at https://github.com/chelof100/mcp-governed-agents.

References

[1] Anthropic. Model context protocol (MCP) specification. https://modelcontextprotocol.io/specification, 2024. Version 2024-11-05.

[2] Marcelo Fernandez. Closing the execution gap in LLM agent systems: Empirical evidence for compliant drift, partial observability, and integrated runtime governance. arXiv:TBD, 2026. Paper 7 of the Agent Governance Series. https://doi.org/10.5281/zenodo.19929771.

[3] Marcelo Fernandez. Identity-bound governance under execution uncertainty: Principal sets, authority resolution functions, and accountability proof blocks for post-HALT re-authorization. arXiv:TBD, 2026. Paper 8 of the Agent Governance Series.

[4] Marcelo Fernandez. Non-blocking governance: Escrow-based asynchronous authorization for human-mediated agent oversight. https://doi.org/10.5281/zenodo.20214654, 2026. Paper 10 of the Agent Governance Series.

[5] Google DeepMind. Agent-to-agent (A2A) protocol specification. https://google.github.io/A2A, 2025. Draft specification.

[6] LangChain AI. LangGraph: Building stateful, multi-actor applications with LLMs. https://github.com/langchain-ai/langgraph, 2024.

[7] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv:2308.08155, 2023.

[8] CrewAI. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. https://github.com/crewAIInc/crewAI, 2024.

[9] OpenAI. Swarm: An educational framework for multi-agent orchestration. https://github.com/openai/swarm, 2024.

[10] Martin Leucker and Christian Schallhart. A brief account of runtime verification. *Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009.

[11] Klaus Havelund and Grigore Roşu. Efficient monitoring of safety properties. In *International Journal on Software Tools for Technology Transfer*, volume 6, pages 158–173, 2004.

[12] Ylies Falcone, Klaus Havelund, and Giles Reger. Tutorial on runtime verification. *Engineering Dependable Software Systems*, pages 141–175, 2013.

[13] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. Design patterns: Elements of reusable object-oriented software. 1995. Proxy pattern, Chapter 4.

[14] Marcelo Fernandez. Atomic decision boundaries in delegated agent systems. arXiv:2604.17511, 2026. Paper 0 of the Agent Governance Series. https://doi.org/10.5281/zenodo.19670649.

[15] Marcelo Fernandez. Agent control protocol (ACP): A formal framework for stateful admission control in autonomous agent systems. arXiv:2603.18829, 2026. Paper 1 of the Agent Governance Series. https://doi.org/10.5281/zenodo.19672575.

[16] Marcelo Fernandez. From admission to invariants: Measuring deviation in delegated agent systems. arXiv:2604.17517, 2026. Paper 2 of the Agent Governance Series. https://doi.org/10.5281/zenodo.19672589.

[17] Marcelo Fernandez. Irreducible governance structure: Fair allocation, composition, and enforcement in multi-agent systems. arXiv:TBD, 2026. Paper 3/4 of the Agent Governance Series. https://doi.org/10.5281/zenodo.19708496.

[18] Marcelo Fernandez. Reconstructive authority model (RAM): Execution validity under partial state observability. arXiv:2604.22898, 2026. Paper 5 of the Agent Governance Series. https://doi.org/10.5281/zenodo.19669430.

[19] Marcelo Fernandez. Operationalizing reconstructive authority: Runtime construction, dependency resolution, and execution gating. arXiv:TBD, 2026. Paper 6 of the Agent Governance Series. https://doi.org/10.5281/zenodo.19699460.